# DevOps Consulting Competency Technical Controls Calibration Guide

## Table of Contents


*DevOps Consulting Competency Technical Controls Calibration Guide* ..... *1*

*Introduction* ..... *4*

*DevOps Case Study Qualification* ..... *5*

*DevOps Case Study Example* ..... *5*

*PROC-001 - Customer Assessment of Internal Organization* ..... *7*

- **Requirement** ..... 7
- **Criteria for Passing** ..... 7
- **Why is this important?** ..... 7
- **How can you implement this?** ..... 7
- **Good Example Response from Globant** ..... 7
- **Unacceptable/Insufficient Information** ..... 8

*PROC-002 - Methodology for Organizational Change Recommendations* ..... *8*

- **Requirement** ..... 8
- **Criteria for Passing** ..... 8
- **Why is this important?** ..... 8
- **How can you implement this?** ..... 9
- **Good Example Response** ..... 9
- **Unacceptable/Insufficient Information** ..... 9

*PRIAC-001 - Templated Infrastructure Provisioning* ..... *9*

- **Requirement** ..... 9
- **Criteria for Passing** ..... 10
- **Why is this important?** ..... 10
- **Good Example Response** ..... 10
- **Unacceptable/Insufficient Information** ..... 11

*PRIAC-002 - Configuration Management* ..... *11*

- **Requirement** ..... 11
- **Criteria for Passing** ..... 11
- **Why is this important?** ..... 11
- **How can you implement this?** ..... 11
- **Good Example Response** ..... 11
- **Unacceptable/Insufficient Information** ..... 12

*PRIAC-003 - Policy as Code* ..... *12*

- **Requirement** ..... 12
- **Criteria for Passing** ..... 12
- **Why is this important?** ..... 12
- **How can you implement this?** ..... 12
- **Good Example Response** ..... 13

Unacceptable/Insufficient Information ..... 13

*PRCICD-001 - Software Release Workflows* ..... *13*
Requirement ..... 13
Criteria for Passing ..... 13
Why is this important? ..... 14
How can you implement this? ..... 14
Good Example Response ..... 14
Unacceptable/Insufficient Information ..... 14

*PRCICD-002 - Build and Test Code* ..... *14*
Requirement ..... 14
Criteria for Passing ..... 15
Why is this important? ..... 15
How can you implement this? ..... 15
Good Example Response ..... 15
Unacceptable/Insufficient Information ..... 16

*PRVCL-001 - Source Control* ..... *16*
Requirement ..... 16
Criteria for Passing ..... 16
Why is this important? ..... 16
How can you implement this? ..... 16
Good Example Response ..... 17
Unacceptable/Insufficient Information ..... 17

*PRMSC-001 - Container Services/Serverless Computing* ..... *17*
Requirement ..... 17
Criteria for Passing ..... 17
Why is this important? ..... 17
How can you implement this? ..... 18
Good Example Response ..... 18

*PRMLO-001 - Cloud and Network Monitoring* ..... *18*
Requirement ..... 18
Criteria for Passing ..... 19
Why is this important? ..... 19
How can you implement this? ..... 19
Good Example Response ..... 19
Unacceptable/Insufficient Information ..... 19

*PRMLO-002 - Distributed Tracing* ..... *20*
Requirement ..... 20
Criteria for Passing ..... 20
Why is this important? ..... 20
How can you implement this? ..... 20
Good Example Response ..... 20
Unacceptable/Insufficient Information ..... 20

*PRMLO-003 - Activity and API Usage Tracking* ..... *21*
Requirement ..... 21
Criteria for Passing ..... 21
Why is this important? ..... 21

**How can you implement this?** ..... 21
**Good Example Response** ..... 21
**Unacceptable/Insufficient Information** ..... 22

***PRPAS-001 - Orchestration to Run and Manage Web Apps*** ..... ***22***
**Requirement** ..... 22
**Criteria for Passing** ..... 22
**Why is this important?** ..... 22
**How can you implement this?** ..... 22
**Good Example Response** ..... 23

***PRSEC-001 - Communication of DevSecOps Best Practices*** ..... ***23***
**Requirement** ..... 23
**Criteria for Passing** ..... 23
**Why is this important?** ..... 24
**How can you implement this?** ..... 24
**Good Example Response** ..... 24
**Unacceptable/Insufficient Information** ..... 24

***Resources*** ..... ***25***

***Notices*** ..... ***25***

# Introduction

This calibration guide is intended for AWS services path partners who have applied or are interested in the Amazon Web Services (AWS) DevOps Competency program. This guide only covers controls under the section of "AWS DevOps Specific Requirements" and the "Common Customer Example Requirements" are addressed in a separate guide.

The calibration guide has case study qualification, case study examples, and deep-dive for each technical control. It provides clarity on the expected level of details for requested evidence. It helps partner improve application quality and reduce cycle time during the technical validation process. Additionally, partners can use the best practices in this technical guide to improve their DevOps service offerings.

Each control has the following FAQs:

**Why is this important?**
This section explains why a particular control is essential to be implemented from an architectural point of view for efficient DevOps operation, security, etc.

**How can you implement this?**
This section discusses how to implement the specific control using AWS services. Partner can implement a certain control using third party services; however, they should be able to justify that the service is adhering to the standards of AWS and meeting the control.

**What are good example responses (If applicable)?**
This section provides good response examples that meet the control and displays the level of depth and expertise required in the assessment.

**What are unacceptable/insufficient information responses (if applicable)?**
This section is composed of response examples not meeting the requirement of the control.

# DevOps Case Study Qualification

Partners need to provide 4 qualified DevOps case studies as part of the Competency application. Use cases focus on just delivering CI/CD projects to customers do not qualify. Instead, we are looking for case studies which transform customer's DevOps practice/approach to AWS Cloud-based infrastructure as code. These projects involve business transformation and technical process transformation. Partners need to demonstrate established DevOps consulting best practice and process documents for customer engagement. Specifically,

- Case study leverages the AWS DevOps Reference Architecture for the 4 stages of the deployment pipeline – source, build, test, prod
- Case study leverages AWS compute, storage, or network services and essential cloud management toolsets such as AWS CloudWatch for resource monitoring and AWS CloudTrail for logging in conjunction
- Case study uses key AWS services AWS CodePipeline, AWS CodeBuild, AWS CodeDeploy, AWS CodeStar, AWS CodeCommit, Amazon Elastic Container Service, AWS Lambda, AWS CloudFormation, AWSOpsWorks, AWS Config, Amazon EC2 Systems Manager, AWS Elastic Beanstalk, AWS X-Ray
- Case study can leverage 3rd party tools for components of development cycle including Configuration Management, Container Services, and CI/CD

# DevOps Case Study Example

**Infrastructure as a Code (IaC) example**

The application does not handle spikes in user traffic and leads to 3-4 outages every month. This resulted in poor customer experience and loss in revenue. Standardize and automate infrastructure creation with infrastructure as code using Terraform. All infrastructure changes go through a CI/CD pipeline that applies quality, security and policy checks on any infrastructure changes. Automated the AMI creation for all the 30 applications that make up the website. We were then able to implement auto scaling using the "Golden AMIs". Auto-scaling was configured with a dynamic scaling policy that uses the Amazon CloudWatch CPUUtilization metric to add capacity when the instance exceeds 90 percent utilization for a specified length of time. On an average there were 2-3 issues with every release related to configuration differences between prod and non-prod. All the non-prod and prod environments were standardized by IaC. This led to zero production issues related to misconfigurations. Implemented auto-scaling with a dynamic scaling policy that uses the Amazon CloudWatch CPUUtilization metric to add capacity when the instance exceeds 90 percent utilization for a specified

length of time. The auto-scaling capabilities of the infrastructure led to a reduction of cloud spend by 200K per month as compute is dynamically allocated based on customer demand.

**Continuous Integration & Continuous Delivery Example**

Customer is unable to meet the business demand to deliver features faster. Current development and delivery process follow waterfall approach where quality and security checks are executed as a final check before changes are deployed to production. Any issues caught at this time are expensive to fix and some changes are pushed accepting the risk. About 80% of the production issues were preventable. AnyCompany implemented a DevOps pipeline using AWS CodePipeline. The pipeline has integrated quality (unit, functional, API, UI, performance) and security (software composition analysis, Static code analysis, Dynamic security testing at runtime) checks. These tests are triggered at every stage in the pipeline DEV, QA, Staging and Production. Few checks like passwords/access key scans and few security checks happen early at the developer IDE. Implemented parallel test execution against ephemeral environments on AWS. Zero development related issues in production Release cycles significantly improved from quarterly to weekly. Improved developer productivity as they don't have to spend time debugging unnecessary issues. Build ran 10 times faster; improved from 60 mins to 6 mins as tests were executed in parallel 60% reduction on non-prod environment cost as we implemented ephemeral environments.

# PROC-001 - Customer Assessment of Internal Organization

## Requirement

AWS Partner evaluates customer's internal organizational structure as basis for providing organizational recommendations to move to DevOps approaches and methodologies.

## Criteria for Passing

Evidence must be in the form of documented approach to evaluate internal organizational structures as part of project risk assessment.

## Why is this important?

Evaluating customer internal organizational structure helps identify any barriers or bottlenecks that hinder the adoption of an agile mindset and cross-functional collaboration. By understanding the current structure and potential risk, you can work with customers to breakdown silos and make necessary adjustments for successful DevOps implementation.

## How can you implement this?

- Align your assessment approach with DevOps principles and define criteria such as collaboration and communication, agility and empowerment, automation and tooling, CI/CD capabilities, and cross-functional integration.
- Conduct assessment on customer organization's current structure, processes, and culture. Consider workshops, interviews, surveys with various placeholders to identify any gaps where DevOps practices can be approved.
- Analyze the assessment results for common challenges or pain points, as well as any strength or existing DevOps practice.
- Develop actionable recommendations for improving organizational structure and prioritize recommendations based on their potential impact and feasibility.

## Good Example Response

AnyCompany has an internal methodology to help transforming organizations to DevOps (partner attached documents/links). we have defined a process to assess organizations and understand the actual state (as-is), diagnose their process, tools and

teams to present future state (to-be) with the principal objective of change the way that the company develop, test, protect and deploy its software. This assessment classified organizations in maturity categories (reactive-proactive-predictive) and levels (baseline - repeatable - consistent - managed - optimized) based on 3 aspects: Culture, process and tech. This way can define and action plan with specific roadmap for working hand by hand with them.

Internally we also have a structure for approaching different kind of sizes and complexity and based on this define for small teams a focal DevOps role in a pod, for organizations with more teams we define a specific DevOps Pod centralizing all the need of other pods/teams, for complex and bigger organizations we define a DevOps chapter and communities with the objective to synchronize technologies and ideas in a common way.

### Unacceptable/Insufficient Information

- A simple response of general customer assessment description without DevOps relevance is not acceptable
- A response without standalone documents (pdf, word documents, links) is not acceptable.

# PROC-002 - Methodology for Organizational Change Recommendations

### Requirement

AWS Partner has a methodology and processes to recommend organizational changes, including identification of plans for various departments (Legal, Finance, HR, and Sales) and addressing the importance of executive sponsorship.

### Criteria for Passing

Evidence must be in the form of organization change recommendations provided to customers and description of outcomes of those projects.

### Why is this important?

Change management process help mitigate risks associated with implementing changes in customer's software delivery process. Having necessary approvals, reviews

and compliance checks help customers meet their regulatory or governance obligations.

### How can you implement this?

- Develop a roadmap that outlines the steps and initiatives required for the DevOps transformation. Consider any potential challenges/resistance to change and develop strategies to address them.
- Communicate the roadmap vision of the DevOps transformation to all stakeholders by explaining the benefits, rationale and expected outcomes.
- Provide trainings and resources (workshops, seminars, hands-on sessions) on relevant tools, processes, and best practices.
- Implement changes incrementally, starting with pilot or less complex projects. Use KPIs aligned with DevOps goals such as deployment frequency, lead time, and customer satisfaction, etc to identify success and/or areas of improvement when scaling up the change implementation.

### Good Example Response

AnyCompany has a Change Management practices as a framework on the delivery of Dev Ops projects (*partner attached documents*). This framework addresses changes at organizational, program and individual level and ensures various stakeholder teams are aligned, trained/empowered thereby accelerating business adoption.

Additionally, we work with customer to build with the client a Change management process during the development life cycle based on assessment of customer's internal organization. This helps to identify the scope of a change initiative, and a single threaded owner to align cross-functional resources and champions of the change objective. Ultimately, it led to organization efficiency with DevOps projects. (*partner attached one customer example detail*)

### Unacceptable/Insufficient Information

A simple response of change management description without DevOps relevance is not acceptable.

# PRIAC-001 - Templated Infrastructure Provisioning

### Requirement

Developers and systems administrators should have an easy way to create and manage a collection of related AWS resources, provisioning and updating them in an orderly and predictable fashion.
AWS Partner has methodologies leveraging templated infrastructure provisioning for repeatable deployments. This can include using AWS CloudFormation or leveraging a Technology Partner solution to provision, secure, connect, and run solution infrastructure.

## Criteria for Passing

Evidence must be in the form of CloudFormation templates or Technology Partner equivalent leveraged for the four (4) submitted customer case studies with description as to how they communicate the need and use of these solutions for repeatable deployments.

## Why is this important?

Templated/automated infrastructure provisioning improves efficiency, reliability, and consistency in managing and deploying infrastructure resources for your customer workload.

### How can you implement this?

- Choose AWS CloudFormation or Terraform to define your infrastructure resources such as EC2 instances, networking components, databases, and other AWS services.
- Utilize parameters and variables within the CloudFormation template or other tools to make it flexible and reusable. Store templates in a version control system like Git to track changes and manage different versions
- Leverage AWS CloudFormation Sample Templates for your application(s).

## Good Example Response

AnyCompany has a repository of CloudFormation templates for our AWS Dev&Ops customer engagement. As a DevOps best practice, every cloud project has to provision IT resources as code from a centralized repository (Infrastructure as Code Reuse Library). This has repeatable and reusable solutions from developer community. We have a variety of resources for systems administrators and developers to deploy to AWS, including a knowledge center, Online Infrastructure as code Community, user forum and wiki. For every business case of our customers, we have separated Dev, Test and Production environments. Please see examples of CloudFormation templates for the four submitted customer case studies.

### Unacceptable/Insufficient Information

A simple response of infrastructure provisioning description without templates used for submitted customer examples is not acceptable.

# PRIAC-002 - Configuration Management

### Requirement

AWS Partner has process to automatically collect software inventory, apply OS patches, create system images, and configure Windows and Linux operating systems. If EC2 Systems manager is not leveraged, Partner can show documented processes and provide technical demonstration for how configurations and infrastructure updates are managed, and the methods used to automate these functions.

### Criteria for Passing

Evidence must be in the form of technology demonstration and process documentation provided to customer as part of their DevOps conversion.

### Why is this important?

Configuration management is crucial for DevOps practices as it ensures consistency and reproducibility. Helping customers implement configuration management facilitates reliability and agility in their software delivery process.

### How can you implement this?

- Identify configuration items and desired state for each items of the infrastructure components. This may include networking configurations, security groups rules, software versions, and other relevant parameters for EC2 instances and other AWS services.
- Use AWS configuration management tools such as AWS Systems Manager, AWS Config or third-party tools to enforce and monitor configuration compliance.
- Continuously review and update your configuration management to keep up with changes in your infrastructure and business requirements.

### Good Example Response

AnyCompany has developed an automated process for configuration management. Automated configuration management practices have been deployed in the customer

examples X, Y. We use AWS Systems Manager to collect software inventory data, apply OS Patches and run maintenance tasks using SSM Automation Documents. The basic configuration of the Systems Manager is deployed via Landing Zone / Control Tower. It deploys CloudFormation Templates which sets up the resource data sync to a central account / bucket, maintenance windows for automated Instance patching via Instance Tags. The attached document is what we provide to the client about best practices and AWS tooling for guiding how to manage configurations and infrastructure updates in our Configuration Management Practices.

### Unacceptable/Insufficient Information

A simple configuration management description without process documents or workflow demonstration (snapshot) is not acceptable.

# PRIAC-003 - Policy as Code

### Requirement

Partner has processes and methodologies to conduct AWS resource inventory, configuration history, and configuration change notifications to enable security and governance. If AWS Config is not used, AWS Partner can show documented processes and provide technical demonstration of how policies are managed, and the methods used to automate these functions.

### Criteria for Passing

Evidence must be in the form of technology demonstration and process documentation provided to customer as part of their DevOps conversion.

### Why is this important?

With policy as a code, customers can continuously monitor and enforce compliance throughout software delivery lifecycles. Any policy violations can be identified early, enabling prompt remediation and reducing compliance risks without relying solely on manual checks.

### How can you implement this?

- Identify the policies you want to implement by considering the compliance standards, security requirements and governance objectives relevant to your customer’s DevOps application

- Choose Policy as code tools such as AWS Config Rules, CloudFormation Guard, and other tools can be used to manage policies.
- Define, validate, and deploy policy rules to enforce access control, resource configurations, and compliance requirements with conditions and actions for the specifics resources.
- Set up automated policy evaluation and remediation actions. Configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic.

### Good Example Response

AnyCompany has mature process for policy as a code where we use AWS Config to set up and monitor the compliance status of AWS resources. Specifically, we set up AWS Config rules for automatic remediations for violations like unencrypted S3 Buckets or unrestricted Remote Access in place. We send the configuration snapshots to a central log archive account which it can be further analyzed. The account owner takes further steps to review and fix the found config violations if needed. Additionally, we use Landing Zone / Control Tower to rollout things like Config and CloudTrail to all account and use SCPs to ensure that those settings can't be altered or disabled.

We share the best practices on how to set up and use AWS Config with our clients. See attached document and screenshots on how AWS Config is implemented. (Partner attached documents).

### Unacceptable/Insufficient Information

A simple policy as code description without process documents or demonstration (snapshot) is not acceptable.

# PRCICD-001 - Software Release Workflows

### Requirement

AWS Partner has processes to build, test, and deploy code every time there is a code change, based on pre-defined release process models. AWS Partner may leverage AWS CodePipeline or equivalent.

### Criteria for Passing

Evidence must be in the form of process documentation describing software release process and examples of how AWS Partner teaches customer how to build software

release models. must be in the form of process documentation describing software release process and examples of how AWS Partner teaches customer how to build software release models.

### Why is this important?

Standardizing the software release workflows reduces risks of errors and inconsistencies, improve overall quality and reliability of the software. Automating the steps in release process leads to increased efficiency and faster time to market.

### How can you implement this?

- Decide on the deployment strategy that suites the application such as blue-green deployments, canary deployments, or rolling deployments by considering downtime tolerance, scalability, and risk mitigation.
- Determine the structure of your release pipeline. Leverage tools such as AWS CodePipeline to connect the stages of your release workflow.
- Map out the flow of the pipeline including code repositories (AWS CodeCommit, GitHub, etc), build and test stages (AWS CodeBuild), and deployment targets (AWS CodeDeploy).

### Good Example Response

AnyCompany has a defined release management process (see attached documents) to manage the software lifecycle transition from development to launching new release with speed and reliability. We developed an internal pipeline workflow management capability using AWS CodePipeline and CodeCommit to facilitate Continuous Integration and Continuous Delivery (CI/CD). See attached release workflow example and backlog release management metrics provided to customer.

### Unacceptable/Insufficient Information

A description on software release workflow without process documents or how it is used in customer example is not acceptable.

# PRCICD-002 - Build and Test Code

### Requirement

AWS Partner has processes to compile source code, run tests, and produce software packages that are ready to deploy. As part of this process, AWS Partner has processes that support provisioning, managing, and scaling servers according to needs. AWS Partners may leverage AWS CodeBuild or equivalent.

## Criteria for Passing

Evidence must be in the form of process documentation describing development and testing processes and examples of how AWS Partner teaches customer how to accomplish the same.

## Why is this important?

Having a process to build and test code enables early bug detection, ensures code quality and supports continuous integration and improvement.

## How can you implement this?

- Configure build environment by selecting a build tool such as AWS CodeBuild, Jenkins, or others and defining a build specification file including dependencies, environment variables and build scripts.
- Set up build pipeline using AWS CodePipeline or Jenkins. Define the source stage to connect to your source code repository and configure build stage.
- Choose a testing framework that is compatible with the build tool. Include test commands or scripts in your build specification configuration and configure the build tool to generate test reports.
- Set up test monitoring using tools such as AWS CloudWatch and integrate test results with feedback system or issue tracking systems.

## Good Example Response

AnyCompany’s DevOps best practice is based on "Infrastructure As Code" (Ansible and CDK) to compile, test and deploy software components automatically. Our core Continuous Integration and Continuous Delivery (CI/CD) processes have the automation to build new development when merging code in the source control repository. The source code for automation (Jenkins pipelines, Ansible, CDK) is stored in "git" versioned. Also, this process triggers quality assurance with automated test like static code review, security code review, unit test, among other testing practices. These ensures we are delivering the best possible customer experience for their DevOps workloads. Attached is a document of how the build & test automation framework is presented and applied to customer example.

### Unacceptable/Insufficient Information

A simple description on how partners build and test code without process documents or how it is used in customer example is not acceptable.

## PRVCL-001 - Source Control

### Requirement

AWS Partner has expertise in implementing source control as part of DevOps practice.

### Criteria for Passing

Evidence must be in the form of demonstration and/or detailed description of how AWS Partner sets up source control systems for customers transforming to DevOps approaches.

### Why is this important?

Source control systems provide version control capabilities, allow developers to work on different features concurrently, and maintain a clear history of all modifications.

### How can you implement this?

- Choose a version control system such as AWS CodeCommit, GitHub, Bitbucket, or others that aligns with your customer's requirements like features, integrations, scalability and pricing. Leverage AWS CodeCommit for a fully managed service with tight integration into AWS ecosystem.
- Set up a new repository in the chosen version control system by defining the name, access controls, and other settings. Clone it to your customer's local development environments with the repository URL so that they can work with the code locally and synchronize changes with the remote repository.
- Make code changes, stage, command, and push changes with the version control system's commands or GUI tools such as 'git add', 'git commit', and 'git push'. Regularly pull the lasted changes to local repository use commands like 'git pull'.
- Determine the branching strategy for customer projects including using main/master for stable code and feature branches for new development. Create and merge the necessary branches as needed with git branch or git merge.

- Monitor and review code changes. Set up notifications or integrations with tools like AWS SNS or AWS Lambda about code changes, commits or pull requests.

### Good Example Response

AnyCompany has source control management as a practice for all IT implementations. We use a variety of source control systems including Bitbicket, GitHub as wells as CodeCommit. Project teams maintains these resources in the version control model and educate customers with each asset update. This allows customers to track changes over time, reducing risk while providing continuity and flexibility. We provided a document on how to set up source control systems including AWS CodeCommit repository according to customers' preferences in terms of technologies. We recommend best practice, branching strategy and tooling based on customer development methodology and product strategy. (partner attached document link)

### Unacceptable/Insufficient Information

A simple description on source control without process documents or demonstration (snapshot) of how it's used for customers is not acceptable.

## PRMSC-001 - Container Services/Serverless Computing

### Requirement

AWS Partner has evaluated and has methodologies to build and deploy microservices architectures leveraging containers or serverless computing.

### Criteria for Passing

Evidence must be in the form of documentation of AWS Partner's approach and philosophy leveraging serverless computing and/or containers with customer examples if/where applicable.

### Why is this important?

Customers can achieve efficient resource utilization and cost optimization by leveraging container services/serverless computing, especially for complex applications needed to be broken into independent components.

### How can you implement this?

- Define a strategy for containerization or serverless development. Identify your customer applications or components of the infrastructure that can benefit from containerization or serverless computing. Consider workload characteristics, scalability requirements, resource usage patterns, and operational issues.
- For Containerization, package your applications into containers using technologies like Docker. Evaluate container orchestration options like Amazon ECS, EKS or AWS Fargate. Leverage Amazon Elastic Container Registry (ECR) and its public gallery for container images.
- For serverless computing, decompose your application into small, self-contained functions that can be executed independently. Develop serverless functions and write code for each function.
- Deploy and manage microservices. Use AWS CloudFormation or AWS CDK to define and provision resources for container deployment. Incorporate it into the CI/CD process of your DevOps best practices.

### Good Example Response

AnyCompany evaluates customer requirements and decide if it is suitable to use containers or serverless computing. Autocaling and pay per use pricing model are attractive capabilities we communicate to customers about the benefits of Serverless Architecture. At AnyCompany we have defined different architecture patterns, including event-driven and microservices strategies to consume serverless components like API Gateway, Lambda, Event Bridge, DynamoDB, SQS, AWS Fargate and others. For containerized applications, we have a defined architecture pattern using AWS EKS and AWS ECS container orchestration services. (see customer example A, B for more details). We recommend containerized application when portability and full control of the environment is a high priority, especially for customers with migrating legacy applications to the cloud. Both approaches are not mutually exclusive, they can complement each other. For instance, if an application function requires more memory than allotted by the serverless component, a hybrid architecture enables developers to leverage the benefits of serverless while still using containers for the functions serverless cannot support.

## PRMLO-001 - Cloud and Network Monitoring

### Requirement

AWS Partner has methodologies leveraging essential cloud and network monitoring services such as Amazon CloudWatch.

## Criteria for Passing

Evidence must be in the form of established monitoring, customer metrics, logs, alarms, and dashboards with description for how/why these items are recommended to customers transforming to DevOps approaches.

## Why is this important?

Implementing cloud and network monitoring into DevOps practice helps customers with infrastructure visibility and identify issues to drive ongoing improvement.

## How can you implement this?

- Define critical metrics that need to be monitored for customer's AWS infrastructure and network and desired monitoring outcomes for customer's DevOps best practice (performance bottlenecks, ensuring high availability, and meeting SLAs).
- Configure monitoring tools including Amazon CloudWatch, AWS VPC Flow Logs, or others to collect metric and logs for all components.
- Leverage CloudWatch insights or integrate with third-party analytics tools to continuous analysis and optimization.

## Good Example Response

At AnyCompany, continuous monitoring is a Dev Ops practice implemented for our customers. We have a specific approach to identify, observe and detect issues and threats during each phase of the DevOps pipeline. Our tool sets include Amazon CloudWatch, CloudTrail, VPC Flow Logs,Config rules, GuardDuty, IAM Analyzer, Security Hub, KMS logging and third-party services DataDog. We work with customers to customize this based on the maturity level of their continuous monitoring practice (see attached link for AWS health metric, logs, and other tools used for the associated case study).

## Unacceptable/Insufficient Information

A simple description on monitoring without dashboard or snapshots of how it's used for customers is not acceptable.

# PRMLO-002 - Distributed Tracing

## Requirement

AWS Partner has methodologies leveraging AWS X-Ray to Analyze and debug production, distributed applications.

## Criteria for Passing

Evidence must be in the form of detailed description of use with at least one customer example. If AWS X-Ray is not leveraged, AWS Partner can describe appropriate use cases and alternative ways this is accomplished

## Why is this important?

Distributed tracing provides customer end-to-end visibility into the request flow across complex applications. It also enables root cause analysis during incident management and debugging process.

## How can you implement this?

- Instrument your application by installing AWS X-Ray SDK or use an AWS X-Ray compatible library in your application's code.
- Enable X-Ray tracing and ensure the trace context is propagated across different AWS services. For example, AWS Lambda and Amazon API Gateway can be automatically instrumented with AWS X-ray.
- Analyze the traced data to understand the behaviours and identify performance issues within customer's application, Utilize the X-ray visualization such as service maps and timelines, to pinpoint areas that require optimization.

## Good Example Response

We leverage AWS X-Ray to analyze and debug the production applications across microservices such as Lambda Functions and Step functions via AWS CDK. Additionally, we work with customers to incorporate distributed tracing to their preferred tools such as Splunk, Dynatrace if needed. See attached a customer example where we used a combination of both. (Partner attached snapshots of specific customer example).

## Unacceptable/Insufficient Information

A simple description on monitoring without dashboard or snapshots of how it's used for customer examples is not acceptable.

# PRMLO-003 - Activity and API Usage Tracking

## Requirement

AWS Partner has methodologies for API and Activity usage tracking via AWS CloudTrail to record AWS API calls and deliver log files.

## Criteria for Passing

Evidence must be in the form of two (2) of four (4) customer examples with description for how/why these items are recommended to customers transforming to DevOps approaches.

## Why is this important?

You can help customer make data-driven decisions for capacity planning and billing/cost optimization by leveraging insights and analytics for usage trend/ patterns from API tracking.

## How can you implement this?

- Enable AWS CloudTrail for your AWS account or specific regions where customers application is deployed.
- Configure CloudTrail setting to captures API activity of specific AWS services including S3 bucket where CloudTrail logs will be stored, log file encryption, and CloudWatch logs integration.
- Extract relevant data from CloudTrail logs using AWS CLI, AWS SDKs to gain insights of API activity and track usage patterns. Leverage AWS CloudTrail Insights or 3rd party log analysis tool to analyze API calls, source IP addresses, timestamps, etc.

## Good Example Response

Any Company has a mature methodology and practices for tracking and monitoring applications/APIs. We deploy CloudTrail in every AWS Account we manage via Landing Zone or Control Tower to monitor API Activity for Multi-Tenant services. All Logs are stored in a central AWS Account where it can be analyzed with QuickSight. The CloudTrail Logs are also replicated into a CloudWatch log group and all activities are

integrated into GuardDuty and Amazon Inspector. Findings are push to our centralized event management tool, allowing notification and elevation notices to customer stakeholder teams. We provided some details/snapshops of CloudTrail events and API requests related with two customer examples (partners attached snapshots).

### Unacceptable/Insufficient Information

A simple description on API usage tracking without dashboard or snapshots of how it's used for customer examples is not acceptable.

## PRPAS-001 - Orchestration to Run and Manage Web Apps

### Requirement

AWS Partner has deep understanding of and leverages AWS Elastic Beanstalk as an orchestration platform to handle deployments from capacity provisioning, load balancing, auto-scaling to application health monitoring. If an alternate such as Heroku is leveraged, AWS Partner to describe reasons for leveraging alternative(s).

### Criteria for Passing

Evidence must be in the form of demonstration and description of PaaS as part of AWS Partner's DevOps strategy.

### Why is this important?

By leveraging orchestration tools, you can automate and streamline the deployment of customer web applications.

### How can you implement this?

Identify the orchestration platform to run and manage web apps based on scalability, customizability, integration with other AWS services, and complexity of the web applications. To leverage AWS Elastic Beanstalk,

- Bundle your web application's source code, dependencies, and configuration files into a deployable package. This could be a ZIP file or a container image

depending on the platform and technology you are using. Refer to tutorial and samples.

- Create an Elastic Beanstalk environment and configure environment variables, database connections, caching options and logging settings.
- Monitor web application metrics, events, and environment status with Elastic Beanstalk console, APIs, or Elastic Beanstalk CLI after deployment. Leverage the built-in auto scaling and load balancing capabilities to ensure performance health and cost optimization.

### Good Example Response

AnyCompany understands the benefits of PaaS like AWS Elastic Beanstalk where we can orchestrate Web application deployment with just a few clicks. This can free developers from deployment-oriented tasks, such as provisioning servers, setting up load balancing, or managing scaling, especially for customers who don't have lots of Cloud Computing expertise.
We choose AWS Elastic Beanstalk for all PoC work and also for production workloads if it supports the application platform and environment variables required for the customer application. See customer example A for more details. For scenarios where AWS Elastic Beanstalk is not suited for the customer business and/or complex architecture requirements, we use CDK to build templates/platforms for more customized controls on application deployment or server configurations. See customer example A for more details.

## PRSEC-001 - Communication of DevSecOps Best Practices

### Requirement

AWS Partner ensures customers understand AWS security processes and technologies as outlined here.

### Criteria for Passing

Evidence must be in the form of onboarding and educational documents provided to customers that specifically cover customer security considerations in the AWS Partner's environment. In addition, evidence must be provided that the AWS Partner helps encourage "shift-left" of security practices, i.e., incorporating security guidelines and

best practices early in development and as an ongoing part of the development and operations process.

## Why is this important?

DevSecOps best practices help customers reduce the risk of security flaws being introduced to the software and minimizes the effort required to remediate them later.

## How can you implement this?

Leverage enablement resources to build training documents and security practices

- Best Practices for Security, Identity, & Compliance
- Integrating security testing at every stage of the software development process

## Good Example Response

We incorporate security frameworks with a shift left approach, adding activities along the software development lifecycle and through automation for seamless integration of security with the business. From the AWS perspective we implement all kinds of approaches to protect information and mitigate possible damage, from strong identity foundation, enabling traceability, applying security to all layers, automating security and protecting data in transit and at rest with AWS services and best practices configuration. From the consulting perspective we have also developed AWS Well Architected framework review where we cover the security pillar as guidelines and best practices on the cloud adoption journey. We mainly help organizations to adopt security practices by:

a. Build secure software using industry-recognized best practices.
b. Design secure applications by integrating security into the architecture and infrastructure design.
c. Reduce software development costs with security by design, resulting in fewer defects, vulnerabilities, and code fixes during production.

Additionally, in our Quality Engineering Studio, shift left is one of our testing capabilities improving quality as early as possible seeking benefits as: reduce costs, improve quality, improve efficiency and competitive advantage. (partner attached links for documents)

## Unacceptable/Insufficient Information

A response without PDF, screenshot, or attached documents that contains the DevSecOps best practices is not acceptable.

# Resources

Visit AWS Specialization Program Guide to get overview of the competency program.
Explore AWS Specialization Program Benefits to understand partner benefits.
Visit How to build a microsite to understand on building a Practice/solution page
Check out How to build an architecture diagram to build an architecture diagrams.
Learn about Well Architected Framework on Well Architected Website

# Notices

Partners are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided "as is" without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers and partners are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers/partners.